

Python + Flutter Developer — Practical Coding Assessment

Task: Build PDF Editor APIs

1. Objective

Build REST API endpoints that provide PDF editing utilities. This assessment now covers two endpoints, described below:

API A — PDF Language Translator (extracts, translates, and rebuilds a PDF)

API B — PDF Watermark (applies a text watermark to every page of a PDF)

You must be able to demonstrate both working APIs using Postman, **and also via a Flutter app that integrates with them**

API A — PDF Language Translator

Endpoint

POST /api/translate-pdf

Behavior

Accepts a PDF file uploaded by the user.

Accepts the source language of the PDF content (e.g. "en" for English).

Accepts the target language the PDF should be translated into (e.g. "bn" for Bangla, or any other supported language).

Extracts the text from the PDF, translates it from the source language to the target language, and generates a brand-new PDF file containing the translated text.

Returns the newly generated, translated PDF file as a downloadable response.

Required multipart fields

file — the source PDF (required)

source_language — e.g. "en" (required)

target_language — e.g. "bn" (required)

PDF Translate Page

File picker to select a source PDF from the device.

Input/dropdown for source_language and target_language.

A submit button that calls POST /api/translate-pdf with the selected file and languages.

Show a loading indicator while the request is in progress.

On success, download/save the returned translated PDF and let the user open or preview it.

On failure, show a clear error message.

Bangla support

The API must correctly handle Bangla (bn) as both a source and a target language, including proper Unicode rendering of Bangla text in the output PDF (this is the trickiest part — most PDF libraries need a Bangla-supporting font embedded manually).

3. API B — PDF Watermark (simplified)

Endpoint

POST /editor/pdf/watermark

Behavior

Accept a PDF file uploaded by the user.

Accept watermark text + basic style options.

Apply the text watermark onto all pages.

Return a new watermarked PDF as a downloadable file.

Required multipart fields

1. **file** — file, required — Source PDF
2. **text** — string, required — Watermark text, e.g. "CONFIDENTIAL"
3. **position** — string, required — one of: top-left, top-center, top-right, center, bottom-left, bottom-center, bottom-right
4. **opacity** — number, required — 0.0–1.0 (e.g. 0.25)
5. **color** — string, required — hex color, e.g. "#FF0000"

Watermark Page

File picker to select a source PDF from the device.

Text field for watermark text.

Dropdown for position (top-left, top-center, top-right, center, bottom-left, bottom-center, bottom-right).

Slider or numeric input for opacity (0.0–1.0).

Color picker or hex input field for color.

A submit button that calls POST /editor/pdf/watermark with the multipart form fields above.

On success, download/save the returned watermarked PDF and let the user open or preview it.

On failure, show a clear error message.

4. Deliverables

Complete, runnable source code (GitHub repo link, zip file, or similar) covering both API A and API B.
requirements.txt / pyproject.toml listing all dependencies.

A short README.md covering: setup steps, how to run the server, and how to test both endpoints in Postman (include a Postman collection export if possible).

A short write-up (half a page is enough) explaining your design choices — libraries used, how you handled Bangla Unicode text, and any known limitations.

5. Time Limit & Submission

Time allowed: 2 days from receiving this test (mention your preferred deadline here).

Submit via: GitHub repository link or a zipped project folder, sent to [your email / recruitment portal here].